Typesetting proof

Three completed chapters only. This is a design inspection artifact, not the complete edition or its release PDF.

Chapter 4 – Domain Models and State Machines

Atlas API has a service identifier, one assigned task, and a running container. Those are three different objects. If an operator scales the service down while an agent is reporting a successful health check, which statement wins? Reading status as a single mutable string cannot answer that question. The domain model separates what the server wants from what the worker has observed, and its transition helpers constrain how their disagreement can resolve.

Our objective is to explain one replica completely enough to reject an impossible report. We will use the active `pkg/types` definitions, the actual memory control plane, and the lifecycle tests. The presence of a `Deployment` or `Event` struct tells us what data can be represented; it does not establish a durable transaction, delivery guarantee, or production deployment topology. Those need separate implementation evidence.

By the end, you should be able to distinguish intent from observation, trace a replica through valid states, and explain why deployment versions and transition owners matter when rejecting stale reports.

Service Intent and Per-Replica Tasks

A `Service` contains identity, namespace, a specification, lifecycle status, and deployment version. Its specification carries replica count and image along with ports, environment, resources, and policy. A `Task` has its own identity, service reference, optional node/container references, image, version, and execution fields. See the service contract in pinned types and task contract in pinned types. The links are immutable snapshots, rather than statements about another revision.

For Atlas, `replicas: 2` is service intent. It does not create a two-process task. The control plane represents two separate units of work, each with its own assigned node and actual status. A task's version connects it to a

service specification at a particular deployment version. Comparing only `service_id` loses that distinction during an update: old and new replicas can belong to the same service while intentionally running different images.

Service versions are recorded by the memory implementation when creation, update, or scale changes occur. The version store implementation keeps specifications indexed by version. This is implemented history within a process, not evidence of PostgreSQL recovery. The helper assigns a Go struct into the version map; it is not a general deep-copy or immutability mechanism for nested maps and slices. Treat version identity as a domain contract without inventing stronger alias protection. A new process cannot reconstruct maps merely because the map values have stable identifiers. Preserve that distinction whenever a diagram labels something a version store.

The service lifecycle is deliberately narrower than task execution. `active` means the service is eligible for desired-state reconciliation; it does not mean all replicas are healthy. `deleting` records an intention to remove work. `deleted` is terminal for that service identity. The service transition validator permits active to deleting, then deleting to deleted. It rejects an immediate active-to-deleted transition, which would erase the explicit convergence phase.

This also explains why a deleted Atlas service should be replaced by a newly created service rather than revived by overwriting its status. Identity carries history. Reusing an old identity can confuse delayed reports, task ownership, and operator interpretation even if its display name looks convenient. The validator's contract is stronger than that convenience: deleted-to-active is invalid.

Desired Status Versus Actual Status

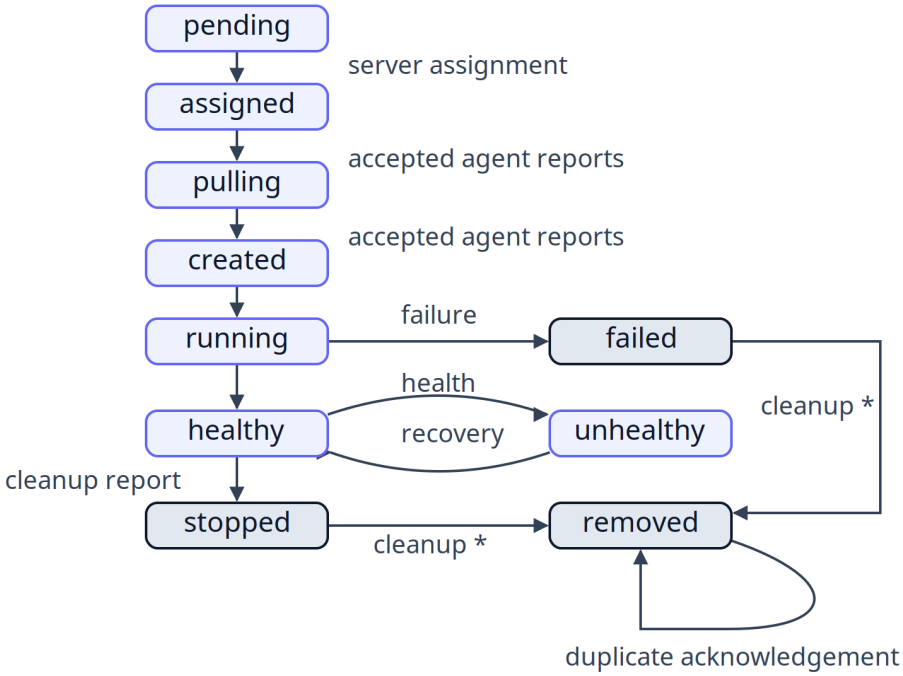
The following observations can all be legitimate moments in one replica's life. They describe a trace through permitted states, not a promise that every polling client will see every intermediate row.

Moment	Server-owned desired status	Actual status	Meaning
Work waiting for placement	running	pending	The requested execution lacks an assignment
Node chosen	running	assigned	A server assignment exists; no process is implied
Worker creates container	running	created	A runtime object exists without a running guarantee
Process observed	running	running	Runtime execution was reported
Health succeeds	running	healthy	A health report accepted at this point
Scale-down requested	removed	healthy	Last observation lags the new intention
Cleanup acknowledged	removed	removed	Task cleanup reached a terminal state

The literal JSON fields are `desired_status` and `actual_status`; neither is interchangeable with service `status`. The complete set of task status constants includes `pulling`, `starting`, `unhealthy`, `stopping`, `stopped`, and `failed` in addition to the table's values. `starting` and `stopping` are reserved progress states in the contract. Their presence does not mean the current agent emits them.

Agent report authorization is a second check beyond membership in the enum. The allowed report list excludes `pending`, `assigned`, `starting`, and `stopping`. The server owns assignment. Agents can report pulling, created, running, healthy, unhealthy, failed, stopped, and removed. This prevents an ordinary worker report from selecting a node or inventing a scheduling transition.

Here is a compact selection of legal actual-state transitions, with ownership written on the arrows. It intentionally omits many legal shortcuts. It is not a complete graph and should not be used to reject an arrow solely because that arrow is absent here.



* Cleanup to removed depends on desired-state checks.
Gray states are terminal for execution.
Selected legal edges; other valid transitions are omitted.

The static diagram has a high-resolution PNG companion at examples/diagrams/task-lifecycle.png for publishing targets that require raster artwork. Gray states are terminal for execution; cleanup to removed remains conditional on desired-state checks.

All names come from the actual-state validator and its permitted branches. `stopped`, `failed`, and `removed` are terminal for execution. The first two permit a cleanup transition to removed in the generic validator; removed has no route back to execution. A health failure is different: unhealthy can recover to healthy or running while failed cannot return to running. Replacement after failure needs a different task identity under the service's restart policy.

Desired status is even more constrained. Its normal progression is running to stopped or removed, then stopped to removed. Same-state updates are allowed. An actual task in failed does not justify changing desired status to failed: server intent and worker observation have different state machines even though their Go field types share `TaskStatus`.

Deployment Versions and Transition Owners

A deployment records from-version, to-version, strategy, status, and rollout limits. It describes an operation over service versions. It is not an alternative container identity. Atlas can therefore have an active service, a running deployment, and both old healthy tasks and new assigned tasks simultaneously. These states answer different questions and should remain separate in dashboards and scripts.

The deployment validator distinguishes pending, running, paused, rolling_back, succeeded, failed, and rolled_back. Succeeded, failed, and rolled_back are terminal. A rollback creates another deployment operation rather than mutating the terminal result of an earlier deployment into success. `paused` being a valid state does not establish an operator-facing pause command; representability and exposed control are separate integration questions.

The same caution applies to node status. Ready, draining, offline, and unknown describe placement eligibility and current observation, not a consensus protocol. Ready can become draining or offline; offline can become ready. A heartbeat timeout can justify recording offline, but it cannot prove that an isolated Docker host has stopped running its containers. The node type has no magical fencing effect.

Events add another record of an occurrence. Their type includes severity, source, related-object identifiers, details, and timestamp. Correlate an Atlas task transition with its task identifier and service identifier rather than reading the event message as a unique key. An event struct does not establish an exactly-once stream. Inspect the mutation and event-emission sites before asserting that duplicate reports cannot duplicate events.

The model developed in two distinguishable steps. The original core-model commit introduced richer types and validation. The lifecycle freeze later centralized transitions and changed API, scheduler, rollout, store, and control-plane callers. The exported patches show these were cross-cutting changes. Original prompts 02 and 29 explain intent, but their numbering is not proof of a one-prompt-one-commit execution history.

There are also older small wrappers under `internal/task` and `internal/node`. They are useful historical context, but they do not supersede `pkg/types`. In particular, using the older node wrapper as the authoritative enum would miss offline. Always trace which type the active API and agent actually serialize.

Rejecting an Impossible Transition

The smallest useful lab requires the locked Go toolchain and disposable application copy, without Docker or PostgreSQL. Start from unchanged source and run the actual transition tests:

```
go test -count=1 -v ./pkg/types -run 'TestValidate.*Transition|
TestAgentCanReportTaskStatus'
```

Our recorded run is `validation/labs/ch04/lifecycle-tests.json`; the exact Go binary, environment keys, UTC times, working directory, exit code, and stdout are retained. It passed. This observation covers the selected unit tests, including rejected pending-to-running and failed-to-running transitions. It does not demonstrate distributed ordering of two real agents or persistence across a process restart.

The domain error is explicit:

```
func invalidTransition(kind string, from any, to any) error {
        return fmt.Errorf("%w: %s %q -> %q", ErrInvalidTransition,
kind, from, to)
}
```

Exact source: pkg/types/lifecycle.go:320–322.

Callers can use `errors.Is(err, types.ErrInvalidTransition)` while retaining a diagnostic that includes the object kind and attempted states. Matching the full formatted message would couple an API client to human wording. Conversely, swallowing the error and updating the task anyway would destroy the contract the helper exists to enforce.

Consider the scale-down failure window. Atlas task actual status is healthy. The server sets desired status to removed. An old healthy report arrives before the worker receives the new intent. AgentCanReportTaskStatus rejects this report because cleanup intent permits only stopped, removed, or failed. A dashboard that blindly takes the last message would revive the replica; the desired-state check prevents that specific inconsistency.

There is a subtle boundary worth testing mentally. The generic validator allows failed-to-removed cleanup, but an agent report also depends on desired state. With desired running and actual failed, the agent gate accepts only the duplicate failed status. With desired removed, a removed report is allowed. A generic transition diagram cannot replace the caller's additional conditions. This is why our diagram annotates terminal cleanup rather than declaring all such reports unconditionally accepted.

For a failure investigation, retain the task identity, desired status, actual status, incoming report, and which helper rejected it. Do not log environment secrets. A rejected report is successful enforcement of this contract, not automatically evidence of a broken agent. The next investigation is whether the agent was delayed, whether desired state changed, or whether a caller submitted an invalid report.

The lab modifies no baseline files, starts no daemon, and creates no containers. Cleanup consists only of optional removal of a separately created test cache; keep the evidence record. A repeated run should use another record filename rather than overwriting the first outcome. If the selected test fails on another version, stop and compare that version's helpers before treating these arrows as current.

Before moving to the API boundary, distinguish three forms of success. A transition helper returning nil proves a proposed local edge is legal. A mutation method completing proves that its own checks and updates completed in that implementation. An agent acknowledging removal proves the reported task reached cleanup, within the evidence available to that server. None alone proves an unreachable host has relinquished a volume, that a previous server transaction survived a crash, or that every observer has received an event. Atlas operators need each statement separately because each has a different failure window and a different owner.

Exercises

1. For a newly assigned Atlas task, list identity, intent, and observation fields needed to correlate a later container report. Explain why service name alone is insufficient.
2. Decide whether unhealthy-to-healthy, failed-to-running, and failed-to-removed are allowed by the generic actual-state validator. Then evaluate the final case through the agent gate with desired running and desired removed.
3. Explain why an active service with a succeeded deployment can still have a failed task. Identify which object a restart replacement changes.
4. An operator proposes accepting healthy reports after desired status becomes removed because they are more recent than the deletion request. Explain the unsafe assumption and give the correct rejection evidence.

Worked answers are in `manuscript/solutions/ch04.md`. They distinguish the helper's local rules from delivery, persistence, and host-fencing claims that require later experiments.

Chapter 9 – Persistence, Transactions, and Optimistic Concurrency

Atlas API has a service ID, desired replica count and several task observations. Those facts currently live in a process. Adding a database beside that process does not answer the operational question: after the process dies, which component reconstructs the same authoritative state? Before promising recovery, we must separate a durable store implementation from an application that actually uses it.

This chapter assumes the executable trace from chapter 6 and the resource model from chapter 5. You will identify the row and transaction that protect an assignment, distinguish conflict from absence, and evaluate a database test without confusing its fixtures with server wiring. The concrete problem is two schedulers trying to assign one pending Atlas task. A second problem, deliberately kept separate, is restoring the service after a new server process starts.

The Store Contract and PostgreSQL Schema

The composite Store interface embeds node, service, task, deployment, event, secret, registry, namespace, quota and GitOps contracts. It is a useful inventory, but not the whole current domain: job, cron job, volume and notification interfaces are declared separately and are not embedded in that composite. This matters when imagining a durable replacement for MemoryService. Satisfying the older Store contract does not establish parity with every method the API currently calls. Claim ch09.contract.

Consider one task row from the initial schema. Its UUID primary key identifies the orchestration task; service_id references a service, and node_id can be null before placement. actual_status records an observation or assignment stage, while desired_status records intent.

The row has the desired service version, creation/update timestamps, restart information and a separate row_version. These identifiers serve different purposes. The service version describes application intent; row_version counts database mutation; neither should be casually relabeled a worker fencing epoch.

The original schema indexes tasks by service, node and status. Those indexes correspond to real lookup patterns: reconciliation asks for service tasks, the agent asks for node assignments, and the scheduler asks for pending work. An index makes a predicate efficient; it does not automatically make a multi-row decision serializable. Foreign keys and CHECK constraints provide another layer: they protect stored relationships and selected values even if a caller bypasses the Go API. They do not validate every lifecycle transition.

Later migrations change this initial model. Namespace uniqueness, image metadata and node agent_version are not present in the first migration. Reading000001 alone therefore describes a historical starting point, not the complete baseline schema. The source lock contains16 up/down pairs. When a test applies a hand-maintained subset, compare that list to fields selected by current SQL before treating a failure as an algorithmic bug.

Timestamps are normalized to UTC at the Go boundary, and SQL updates explicitly set an update time. TIMESTAMPTZ values describe instants rather than a display timezone. Preserve the value returned by the store as the next optimistic token; do not reconstruct it from a formatted dashboard string with lower precision. The current guarded writes compare expected_updated_at, even where an additional integer version is also incremented. A reader must inspect the predicate actually used, not infer it from a column name.

Transactional Boundaries and Conflict Errors

PostgresStore wraps a pool-backed postgresStore. Its WithTx method creates a transaction-scoped store whose db points at the transaction and whose begin field is unset. Operations invoked through that scoped

object participate in the same transaction instead of opening independent nested transactions. The callback's successful return permits Commit; an error returns without committing, and deferred rollback handles the unfinished transaction.

internal/store/postgres.go, lines 69–83 at the locked revision.

```
func (s *postgresStore) withTx(ctx context.Context, fn
func(*postgresStore) error) error {
        if s.begin == nil {
                return fn(s)
        }
        tx, err := s.begin(ctx)
        if err != nil {
                return mapPostgresError(err)
        }
        txStore := &postgresStore{db: tx}
        defer rollback(ctx, tx)
        if err := fn(txStore); err != nil {
                return err
        }
        return mapPostgresError(tx.Commit(ctx))
}
```

This is a meaningful boundary only if every relevant effect uses the transaction-scoped value. Calling the outer pool-backed store from inside the callback escapes that boundary. The generic store.WithTx helper also has a deliberate fallback: if its candidate does not implement Transactor, it simply calls the callback directly. That helps shared controller code work with fakes, but the function name alone cannot prove atomicity for every candidate. Claim ch09.transaction.

Suppose Atlas service creation and its domain event should be one database fact. If service insertion succeeds but event insertion fails, the transaction can roll both back when strict event emission propagates the error. Without that transaction, the service may exist without its corresponding history record. External Docker creation cannot join this SQL transaction; treating it as another callback statement would leave an unavoidable side-effect gap. The orchestration protocol needs reconciliation rather than pretending all systems share a commit.

The inspected transaction audit makes this distinction concrete. Commit 72fd9baff0bbfde49574257380bcd8f747cc2070 adds transaction-aware controller operations and strict events, including a rollback test. Earlier commit 7eba6544ffe2eba41b81faa811e899faf28b77a2 introduced the PostgreSQL layer. Their patches establish code evolution, not a production migration or successful execution of every original development prompt. Prompts 03 and 24 were added retrospectively; prompt 30 describes the later transaction audit.

Conflict handling is also part of the contract. A missing row, a stale optimistic token and an invalid state are not interchangeable. The scheduler treats a lost assignment race as a normal contention outcome and can continue; a transport or SQL error needs a different operational response. Retrying every error indefinitely would hide schema failures and could prevent useful work elsewhere.

Optimistic Writes Under Contention

AssignTask first obtains the current task according to its transaction context, checks whether pending→assigned is legal, serializes allocated ports, and issues a conditional update. The crucial predicate requires the same task ID, expected update time and actual_status pending. The update stores the chosen node and ports together with assigned state. Claim ch09.assignment.

internal/store/postgres.go, lines 912–927 at the locked revision.

```
        row := s.db.QueryRow(ctx, `
                UPDATE tasks
                SET node_id = $2,
                        actual_status = $3,
                        ports = $4,
                        updated_at = timezone('utc', now()),
                        row_version = row_version + 1
                WHERE id = $1 AND updated_at = $5 AND actual_status
= 'pending'
                RETURNING id, namespace, service_id, node_id,
container_id, desired_status, actual_status,
                        image, requested_image,
resolved_image_digest, image_registry, image_name, image_tag,
                        version, ports, conditions, restart_count,
failure_reason, created_at, updated_at, started_at, finished_at`,
                string(id),
                string(nodeID),
                string(types.TaskAssigned),
                portBytes,
                expectedUpdatedAt.UTC(),
```

Imagine schedulers A and B both read task T at time token U. Both compute a valid placement. A's update succeeds and changes T's status and update timestamp. B's later conditional update no longer matches. The acceptable result is one owner and one conflict, not two success responses with different node IDs. Replanning after the conflict is different from blindly repeating B's old write: the new snapshot may show that T is no longer pending.

This protects one task assignment. It does not, by itself, reserve all free CPU and host ports across a set of independently planned tasks. Two schedulers can each read available capacity, plan different pending tasks, and successfully claim both even if their combined plans oversubscribe the snapshot. Chapter 10 separates per-task uniqueness from global reservation. Increasing the number of predicates on one row cannot replace a deliberate cross-row allocation protocol.

Service updates follow a similar pattern: updateService writes the new spec and increments deployment_version only where the service ID, namespace and expected timestamp match. A concurrent operator holding an old token receives ErrConflict rather than silently replacing a

newer specification. That is valuable lost-update protection, but the surrounding operation still needs to connect version history and events atomically. A successful SQL row update is not proof a worker is already running the new image.

Observation	What it establishes	What remains unproved
Conditional assignment rejects stale writer	One row did not accept both old decisions	Global CPU/port serialization
Service/event callback rolls back	Both SQL effects share the tested transaction	Docker-side rollback
Full schema migrations apply	Selected schema can be constructed	Default server uses that schema for live state
Controller restarts with one store object	Repeated controller logic can reuse retained state	Fresh-process recovery

Atlas Lab: Database Evidence Without a Silent Fixture Repair

Use only the disposable source copy and the dedicated labelled PostgreSQL container from the environment contract. The database name must end in _test because the upstream migrate helper refuses other names. The book's additional guard checks the loopback binding, exact database identity and ownership labels before permitting destructive tests. A suffix alone is not sufficient evidence that the database is disposable.

The actual guarded run is preserved in database evidence. WithTxRollsBackOnError passed. Integration and ConcurrentAssignTaskOnce failed with SQLSTATE 42703: current node queries require agent_version, while the reset helper applies only migrations 000001–000014 and the column arrives in000016. This is an executed fixture/schema mismatch. It is not a passing concurrency experiment, nor a reason to erase the failure and rerun altered upstream code under the same label.

```
python3 books/inside-orch/scripts/run_database_tests.py \
  --container <recorded-owned-postgres-container> \
  --working-copy /tmp/inside-orch-labs-cbfc94ade5b5/orch \
  --go /path/to/go1.25 --check-only
```

Run that from the website root; only remove --check-only after the scope check succeeds. Keep the designated URL in the environment rather than in copied commands or logs. The separate full-schema teaching fixture applied all 16 migrations and exercised AssignTask with two concurrent writers: one won and one received a conflict. It also preserved a coherent stored task. Its passing result is named separately from the failing untouched suite, and its fixture remains reviewable.

For a source-only alternative, inspect the conditional predicate and the rollback assertions, recording them as inspected rather than executed database outcomes. Clean up by stopping the exact owned container after all dependent experiments, then removing only its recorded dedicated volume if one was created. Never reset a shared developer database or use a broad volume-prune command.

Why the Default Server Is Still Not Durable

The executable still constructs NewMemoryService with envelope, policy and retention options. That object owns maps. Database configuration and schema checks do not redirect its methods into PostgresStore. The persisted tables can be correct and the default server can still lose its authoritative service state on process restart. This is the boundary that prevents a storage library from becoming an accidental operational promise. Claim ch09.composition.

A durable composition exercise therefore starts with a method inventory, not a constructor substitution. Identify service version history, node credentials, jobs, local volume attachments, notification sinks, operations and quota state. For each, choose the authoritative store and transaction boundary. Then decide how controllers rebuild observations, how agents re-enroll without duplicating work, and how pending side effects are

recovered. Persisting a service row while dropping its ownership or secret metadata can make restart more dangerous than an honestly disposable memory server.

The release decision should require a fresh-process test using an independently constructed application instance and the same durable database. Preserve container IDs and declared intent before termination, start a genuinely new process, and verify the restored API and reconciliation behavior. Do not reuse one MemoryService object and call the result database recovery. Until that composition exists and passes its own tests, describe PostgresStore as an implemented package and the running default service as memory-backed.

A useful contention record includes both input task snapshots and both outcomes, but excludes the database URL and authentication material. Compare the original UpdatedAt with the returned winning task, then re-read the stored row. The losing result alone is insufficient: an implementation that rejected both writers would also produce a conflict without making progress. The full-schema fixture checks that exactly one writer wins and the stored assignment is coherent. This provides positive and negative evidence together. It still says nothing about whether the chosen node has enough remaining capacity, because that is a different invariant with a larger read set.

Exercises

1. Trace one task row through AssignTask. Which predicate rejects a second stale writer, and why is row_version alone insufficient evidence of the guard?
2. Explain how a callback can accidentally escape WithTx even while executing inside its lexical body.
3. Classify the observed missing-agent_version failures. What separate fixture would isolate assignment correctness without rewriting the baseline?

4. Design the minimum evidence needed to claim durable Atlas service restart. Include one API state that is absent from the composite Store interface.

Worked answers. Exact source slices and their checksums are retained in chapter listings. The chapter's claims distinguish inspected implementation, the executed database fixture failure and the still separate durable-composition design.

Chapter 17 – Rolling Updates and Rollback Invariants

Atlas API already has useful replicas. Its next image adds a feature, but the operator must change those replicas without confusing a deployment request with successful replacement. One new container can be created before it is ready, one old task can be told to stop before Docker acknowledges removal, and traffic can still use an endpoint cached before either change. A rolling update is therefore a sequence of observations and bounded decisions, not an atomic exchange of images.

This chapter assumes the task lifecycle, agent report path and health thresholds from Chapters 4, 13 and 14. You will calculate surge and availability bounds, distinguish failed target versions from previous successful versions, and identify conflicting user actions. The concrete mechanism is the rollout controller used by default server startup. Its state and history still reside in MemoryService; learning its count invariants does not remove the process-loss boundary established earlier.

Service Versions and Deployment Ownership

RolloutService changes the service's desired image, increments DeploymentVersion and saves a versioned specification before recording a pending deployment. FromVersion and ToVersion identify the transition. The API returns ownership of an operation that will advance later; it does not wait for every agent to start a container. The controller retrieves deployments in pending, running and rolling_back states, sorting by creation time and ID. This makes iteration order inspectable without making the collection durable.

A deployment is interpreted against current service replicas and tasks. Task.Version matters: an old image running under an old task belongs to a different cohort from a new target. A task from neither deployment

version is ignored by this particular accounting function. That is a reason to retain versioned evidence during triage rather than merely count every container with the service's name.

The source distinguishes active tasks, available old tasks and healthy new tasks. Old availability uses the lifecycle helper accepting running or healthy. New readiness is stricter: only actual healthy increments newHealthy. A new task whose container is merely running occupies active capacity but cannot authorize stopping an old replica. This difference is easy to lose when a dashboard compresses every live task into a green count.

Unchanged source excerpt — internal/rollout/rollout.go:376–384:

```
                if task.ActualStatus == types.TaskFailed {
                        state.newFailed++
                }
                if isActive(task) {
                        state.newActive++
                        if task.ActualStatus == types.TaskHealthy {
                                state.newHealthy++
                                state.available++
                        }
```

The claim here is bounded: this controller uses reported states from its store. It does not independently probe the application or synchronously update every proxy. A routing consumer must also observe the new endpoint set before client continuity can be evaluated. Its arithmetic can be correct while a client still reaches a stale or unreachable endpoint. Keep controller evidence and traffic observations separate throughout the experiment.

Max Surge and Max Unavailable Arithmetic

Take Atlas at three replicas, MaxSurge one and MaxUnavailable zero. The creation ceiling is three plus one: four active tasks. Initially three old tasks are available and there are no targets. One pass may create a target,

filling the fourth slot. Because that target is pending, no old task can stop. Repeating the pass without new reports cannot manufacture progress: the surge slot is already occupied and newHealthy remains zero.

When the first target reports healthy, the controller can stop an old task while keeping three available replicas. Its stop guard calculates the unavailable count after the prospective stop. With four available tasks, stopping one leaves three, so three minus three is zero. A second stop would leave two, making the unavailable count one and violating this deployment's zero allowance. The next observation must establish what actually stopped before capacity can be reused.

Unchanged source excerpt — internal/rollout/rollout.go:244–252:

```
        for _, task := range oldTasks {
                if state.newHealthy <= state.oldStopped+stopped {
                        break
                }
                if service.Spec.Replicas-(state.available-stopped-1)
> deployment.MaxUnavailable {
                        break
                }
                transactional := c.transactional()
                stoppedTask := types.Task{}
```

There is another guard before that arithmetic: newHealthy must exceed oldStopped plus the stops planned in this pass. It relates each removed old replica to replacement readiness. Consequently MaxUnavailable is not a command to stop that many old tasks immediately. For example, a three-replica deployment with surge zero can stall because no target can be created and no healthy replacement exists to authorize stopping. The source's conservative ordering is more specific than a generic rolling-update policy description.

The creation loop tracks totalActive after each successful task write; stopping tracks the same local pass. Neither count turns a distributed runtime observation into a transaction over Docker. A task can be counted active until the store observes its terminal state, and a stop directive can coexist with a container that has not received it. Developers

should identify whether a suspected limit violation concerns desired objects, reported task states or actual live processes before changing the arithmetic.

Atlas observation	Old active	New active	New healthy	Controller consequence
Before update	3	0	0	One surge slot can be filled
Target pending	3	1	0	Wait; no old stop authorized
Target healthy	3	1	1	One old stop can preserve availability
Stop still unacknowledged	Depends on desired/actual active predicate	1	1	Inspect task state rather than assume Docker removal

This table is a source-derived scenario, not a captured execution trace. In a real run, attach the task IDs and timestamps that make each row meaningful.

Failed Targets and Previous Successful Versions

A target in TaskFailed causes the deployment to become failed. TaskUnhealthy is a different state: rolloutStateFor does not increment newFailed for it. The agent reports unhealthy after its threshold; MemoryService.ReportTaskStatus converts that report to failed when the service restart policy permits replacement. Without that conversion, an unhealthy stored target is distinct from failed. The rollout controller does not interpret raw probe responses directly. During the intermediate window the target can occupy a surge slot while progress waits.

Imagine a healthy first target and then a second target whose application returns an error. The first target may already have allowed one old replica to stop. A later target failure marks the rollout failed; it does not

atomically resurrect the stopped container. The remaining old and new cohorts are evidence of a partially advanced deployment. An operator needs their versions, states and endpoints before selecting recovery.

RollbackService finds a previous successful version and restores its specification as the desired target. It records a new deployment from the current version to the earlier one, preserving the failed deployment's history. TestRollbackAfterFailedRollout asserts version two returning to version one and confirms that the failed forward deployment remains failed. TestRollbackWhenNoPreviousVersionExists rejects a rollback without such a version. These are actual source assertions, not proof every runtime image remains pullable.

An in-flight rollback is idempotent: repeated calls return the existing operation rather than generate another reverse transition. That property is valuable when the response to the first request is lost. It is scoped to the same live control plane and operation identity. A new default server has neither the version history nor the prior operation maps, so retrying after process loss is a different situation.

The failed-image timeline should therefore name five events: forward request accepted, target created, target health degrades, deployment becomes failed, reverse deployment accepted. Add a sixth observation for reverse completion only when task reports establish it. Preserving the failed operation allows an incident report to explain why rollback was necessary instead of rewriting history into a smooth success story.

Locks Against Conflicting User Actions

Operation locks are enforced through service/deployment state under the memory service's mutex. An identical repeated rollout can return the active deployment. A different rollout conflicts, and rollback during an active forward rollout conflicts. These rules avoid two callers claiming incompatible desired versions at once. They do not constitute a distributed lock across independent servers.

Deletion has precedence. TestDeleteWinsOverActiveRollout requests delete, checks that the active deployment becomes failed, and rejects a later rollout against the deleting service. TestScaleAndDeleteOperationOrdering shows that scale after delete conflicts while scale before delete can complete before deletion wins. Repeated delete is idempotent. The practical rule is to classify an error as an operation conflict before retrying a different request; a blind retry can obscure the operator's intended order.

A failed deployment is terminal, so the operator can explicitly start rollback after failure. There is no need to invent a force flag merely because the original development prompt mentions one possible policy. Use the actual CLI and control-plane behavior. The lock tests are particularly useful because they make the ordering contract visible without requiring a timing-sensitive terminal demonstration.

Atlas Lab: Separate Count Invariants from Traffic

Use the disposable pinned source copy with Go 1.25 and an owned cache. Run the named deterministic tests first; they need no Docker or database. Their fakes explicitly mark tasks healthy, which is exactly why they can prove controller decisions reproducibly but cannot prove uninterrupted application traffic.

```
go test ./internal/rollout -run 'TestController(SuccessfulRollout|
FailsWhenNewTaskFails|RespectsMaxUnavailable|RespectsMaxSurge|
IdempotentResume|CompletesRollback)$' -count=1 -v
go test ./internal/controlplane -run 'Test(Rollback|
RolloutWhileRolloutInProgress|DeleteWinsOverActiveRollout|
ScaleAndDeleteOperationOrdering)' -count=1 -v
```

Expected observations are the assertions just described: a bounded target cohort, no old stop before replacement health, failed forward history preserved, an idempotent reverse operation and clear conflicts. These commands are source-checked experiment instructions. This chapter's writer does not label them a fresh executed run; attach matching command records before doing so.

For a separate real Atlas run, record initial service version, all task/container IDs and direct HTTP responses. Use the case-study health-enabled manifest and an owned second image. Request rollout with one surge slot, then observe deployment status, task version/status and endpoint changes after each interval. Inject a failing target using the dedicated unhealthy manifest or nonexistent lab image; never modify a user service. Record both the failed deployment and the reverse operation.

Abort if the service name resolves outside the recorded run namespace, if expected old endpoints disappear before the recorded availability permits it, or if the selected failure affects an unrelated container. Cleanup deletes only Atlas resources created by that run and checks residual container IDs. A passing count fixture cannot waive this cleanup or routing evidence. The engineered decision is to trust each observation for the boundary it actually measures.

Exercises

1. Atlas has three old available replicas, one healthy target, surge one and unavailable zero. How many old replicas may stop in the next decision, and which guard prevents the second stop?
2. Explain why a new running target cannot replace an old running replica in this controller even though both appear live.
3. A rollout has one healthy target and one unhealthy target. Is failure status guaranteed immediately? Trace the restart-policy conversion needed and identify the waiting window.
4. The first rollback response disappears. Contrast retrying against the same live server with retrying after a fresh default server process. Which evidence would establish operation identity?

Worked answers provide calculations and source-based reasoning. Claims and exact listings are recorded in the companion evidence ledger; their inspected status must not be upgraded without an actual execution record.

Development history and source references

Original development prompts 14, 15, 42 supply the requested scope. The inspected implementation history includes de5900bced92, 430a061fe7f0, 5bfb4ac394a6. Their exported changed paths were compared with this chapter's implementation. These are topical associations; early prompts were retrospective and later work can share a commit. A tracker entry is not evidence that the described experiment ran.

- internal/rollout/rollout.go
- internal/rollout/rollout_test.go
- internal/controlplane/rollback_test.go
- internal/controlplane/operation_lock_test.go
- docs/ROLLOUTS.md